

I've named my language `kuip` or `kuipscript` (just the first part of my last name).

Regular expressions in kuip:

- Number Literals → `\d+|\d+\.\d+`
 - this allows for standard digits and decimals, it currently doesn't work with negatives
- String Literals → `\"[[:alnum:][:alpha:]\d\(\)\"]+`
 - This includes all letters and numbers, as well as a variety of special characters such as ()
- Identifier Literals → `[[:alnum:][:alpha:]\d\_\-]+`
 - This works similarly to string literals, except it only allows numbers and characters, as well as underscores and dashes

Design Choices Relative to Lox:

When designing this, I followed the textbook guide which used lox as a reference. This led to a lot of my language to use similar functionality to lox, such as function names and other keywords. There are some deviations such as print being say, but with time it will have more unique features that separate it from other languages.

How to run the code:

There are a number of ways to start the code or use the tests

- Running through the terminal
 - Interactive mode:
 - `py src/main.py`
 - Source/file mode
 - `py src/main.py tests/lab1/lab1-testingText`
 - `py src/main.py (input)`
 - Tests:
 - `py tests/lab1/lab1-testingText`

Test Cases:

Test num:

- Purpose:
 - This test checks if the scanner can properly make a single number token
- Input:
 - 123
- Expected Tokens / Errors:
 - It should create a token with type NUMBER and a Lexeme of “123”

- Actual Output:
 - Type: NUMBER, Lexeme: "123" – this is the expected result

Test_nums:

- Purpose:
 - More in depth test of test_num which handles multiple numbers in a single input
- Input:
 - 123 456
- Expected Tokens / Errors:
 - This should create two tokens, each as type NUMBER and both's lexeme being their associated value
- Actual Output
 - 2 Tokens, one is type: NUMBER lexeme: "123" and the other is type:NUMBER lexeme: "456" – This is the expected output

Test_floats:

- Purpose:
 - This tests to see if floating / decimal values are properly recognized in the scanner
- Input:
 - 123.45 678.90 59.99 0.002
- Expected Tokens / Errors
 - There should be 4 tokens, each of which are NUMBERS and have lexeme of their respective values
- Actual Output:
 - Type: NUMBER lexeme: "123.45"
 - Type: NUMBER lexeme: "678.90"
 - Type: NUMBER lexeme: "59.99"
 - Type: NUMBER lexeme: "0.002"
 - All were the expected results

Test_strings:

- Purpose:
 - This test is to see if text / strings are turned into the proper tokens
- Input:
 - "hello" "world"
- Expected Tokens / Errors
 - There should be two tokens, both are type STRING and one is "hello" while the other is "world"
- Actual Output:
 - Type: STRING lexeme: "hello"
 - Type: STRING lexeme: "world"
 - This was the expected result

Test_empty_string:

- Purpose:
 - Tests if an empty string is still identified as a string
- Input:
 - ""
- Expected Tokens / Errors
 - A token with type string and a lexeme of an empty string ""
- Actual Output:
 - Type: STRING lexeme: ""
 - This was the expected result

Test_keywords:

- Purpose:
 - Ensures all unique keywords are properly set on the tokens
- Input:
 - and class else false for fun if nil or say return super this true var while
- Expected Tokens / Errors:
 - Each token should be its associated type (except say which means print)
- Actual Output:
 - Each of the keywords matched their associated type and lexeme values, and say translated to print properly

Test_identifiers:

- Purpose:
 - Tests to make sure that a keyword that isn't part of the list or a string will become an identifier.
- Input:
 - "this_identifier anotherIdentifier _privateVar"
- Expected Output:
 - Each should be given type: Identifier and then lexeme should be a string of themselves
- Actual Output:
 - this_identifier became type IDENTIFIER and the lexeme was "this_identifier". This repeated correctly for the other options too

Test_unexpected_character:

- Purpose:
 - Checks if an invalid character is ignored rather than being added to the tokens
- Input:
 - @
- Expected Output:
 - There shouldn't be any tokens in the list as long as this was the only input
- Actual Output:
 - The @ symbol didn't appear anywhere in the token list

Test_terminated_string:

- Purpose:
 - Makes sure that an incomplete string isn't added to the token list
- Input:
 - "Untimernated
- Expected Output:
 - There shouldn't be any tokens in the list
- Actual Output:
 - The unterminated string wasn't added to tokens

Test_file_input:

- Purpose:
 - Makes sure that the program can open a file to read and scan its contents
- Input:
 - A txt file called [lab1-tests.py](#)
- Expected Output:
 - Each of the items inside of [lab1-test.py](#) are run through the scanner and properly added to tokens
- Actual Output:
 - No errors were raised and each input in the text document were properly translated to tokens

Known Limitations:

As of now the scanner doesn't work with negative numbers which restricts some of the calculations it can do. This feature is planned to be implemented soon, along with more keywords for more functionality.